

Data Classification

Preprocessing a dataset the right way — scaling, encoding, correlation, and dimensionality reduction with PCA

A beginner-friendly walk through the full classification workflow, built around the Vehicle Silhouettes dataset: load the data, encode the target, split it, scale the training features, train a Linear SVC, hunt down correlated features, and shrink them with Principal Component Analysis. Every term from the study list is defined in the glossary at the back.

1 The big picture: what classification is

Classification means teaching a computer to sort examples into **categories**. Feed it many labeled examples, and it learns patterns that let it label new ones. The category we predict lives in the **target column** (also called the **label** or **y**); everything used to predict it are the **features** (the **X** columns). Because our vehicle dataset has four possible answers, this is a **four-class** (multi-class) classification problem.

Real-world uses look the same underneath: predicting stock prices, recommending products on e-commerce sites, or — here — naming a vehicle from the shape of its silhouette. The model is only as good as the data you hand it, so most of the work is **preprocessing**: getting the data into the right shape and scale first.

The classification workflow



optional: PCA (dimensionality reduction)

The steps in this guide. Note the order: we split before scaling and PCA, so those are learned from training data only. Green boxes handle the data; orange boxes train and measure the model.

TIP — Preprocessing is most of the job

A model can't fix messy inputs. Scaling, encoding, and trimming correlated columns often help more than switching algorithms.

2 Load and inspect the data

Data usually arrives as a **CSV** file. **Pandas** reads it into a **DataFrame** — a table of rows (**records**) and named columns. In our set, each record describes one vehicle's silhouette with numeric **features** like *compactness*, *circularity*, *scatter ratio* and *elongatedness*, and the **class column** at the end holds the answer (bus, van, saab, ...).

```
import warnings
warnings.filterwarnings('ignore')      # silence noisy deprecation messages

import pandas as pd
vehicle = pd.read_csv('../datasets/vehicle.csv', sep=',')
vehicle.head()                        # preview the first 5 rows
len(vehicle)                          # 846 records
```

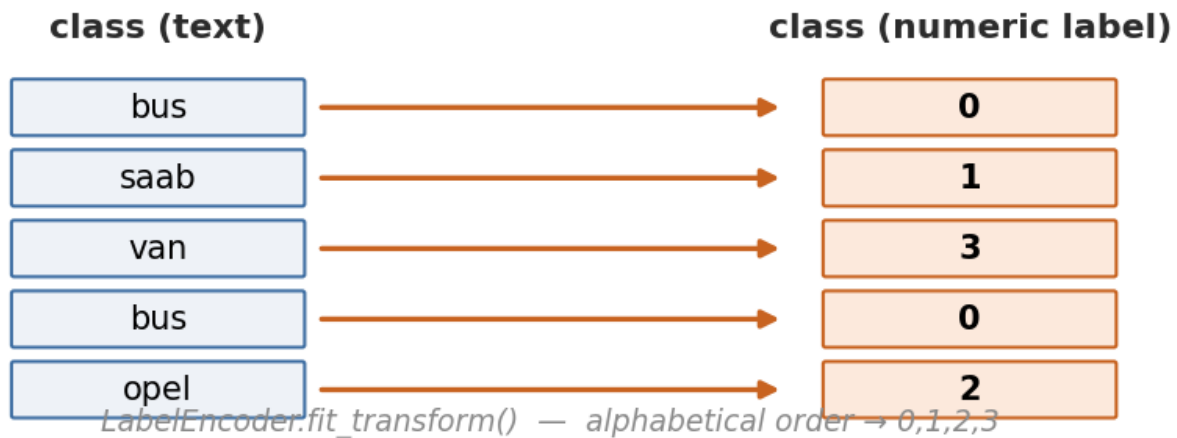
Not every column earns its place. Several silhouette measurements overlap, so we **trim** the dataset with the DataFrame **drop** function (`axis=1` means "drop columns", `inplace=True` edits the table directly):

```
vehicle.drop(['pr.axis_aspect_ratio', 'max.length_aspect_ratio',
              'pr.axis_rectangularity', 'scaled_variance.1',
              'scaled_radius_of_gyration.1', 'elongatedness',
              'skewness_about.1', 'skewness_about.2'],
             axis=1, inplace=True)
vehicle.head()                        # now 9 features + the class label
```

3 Encode the target column

The features are already numbers, but the **class** column holds **strings** (stored as pandas' **object** type). Most algorithms can't read text, so we convert each category to an integer — this is **label encoding**. scikit-learn's **LabelEncoder** assigns numbers in alphabetical order, starting at 0.

```
from sklearn import preprocessing
le = preprocessing.LabelEncoder()
vehicle['class'] = le.fit_transform(vehicle['class'].astype(str))
vehicle['class'].unique()      # array([3, 2, 0, 1]) -> four classes
```



LabelEncoder turns each distinct class name into a whole number. With four classes you get labels 0, 1, 2, 3.

Four classes give us a **benchmark**: if they appear about equally often, pure **random guessing** would be right roughly 25% of the time (the **probability baseline**), and a useful model must beat that comfortably. When the classes are imbalanced, the bar is different — the baseline to beat becomes the share of the single most common class.

NOTE — LabelEncoder is for the target, not the features

Use it on y. For input features, prefer one-hot encoding, because plain integer labels imply a false order (2 is not “twice” class 1).

4 Split into training and test sets

To judge a model honestly, you test it on data it never saw during training. Do the split **early** — before scaling or PCA — so nothing about the test set can leak into how those steps are fit. The **train_test_split** tool carves the data into a **training set** (used to learn) and a **test set** (used only to measure). You could slice the NumPy array by hand, but this one line is cleaner.

```
from sklearn.model_selection import train_test_split
X = vehicle.drop('class', axis=1)      # the 9 feature columns
y = vehicle['class']                  # the encoded target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1)  # split FIRST, fit preprocessing later
```

A horizontal bar divided into two segments. The left segment is green and labeled 'Training set — 80%'. The right segment is orange and labeled 'Test 20%'.

Training set — 80%

Test 20%

```
train_test_split(X, y, test_size=0.2, random_state=1)
```

test_size=0.2 keeps 80% for training and 20% for testing. random_state fixes the shuffle so the split is reproducible.

NOTE — Why random_state?

It seeds the shuffle so you (and your readers) get the exact same split every run — essential for repeatable results.

5 Scale the features

Features measured on different scales can confuse a model — a column ranging 0–1000 can drown out one ranging 0–1. **Scaling** puts them on comparable footing. Because we've already split, we **learn the scaling from the training set only** and then apply those same numbers to the test set. There are two very different directions to scale.

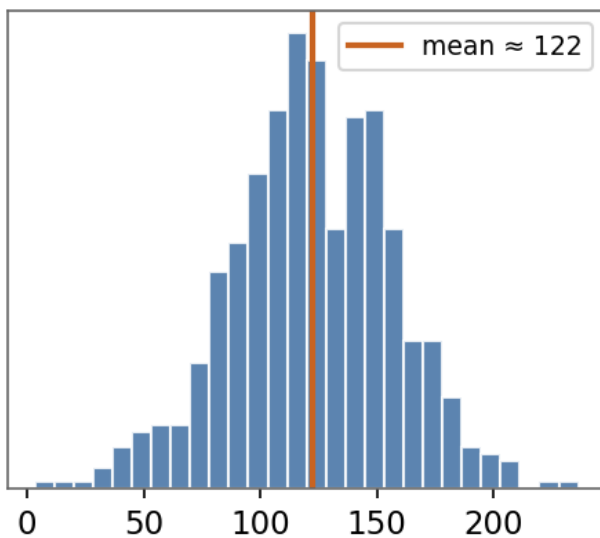
Standardization (column-wise)

A **StandardScaler** works down each **column** independently, shifting it to **zero mean** and **unit variance** (standard deviation 1). This is also called **longitudinal** or **column-wise** scaling. Call **fit** on the training data, then **transform** both sets:

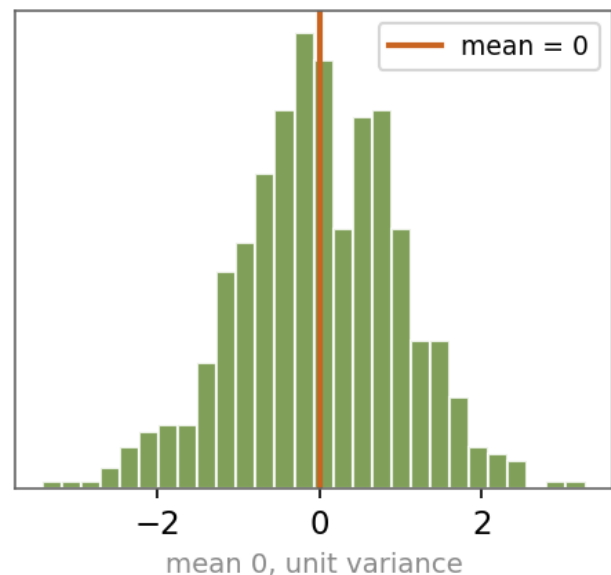
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # learn mean & std on TRAIN only
X_test = scaler.transform(X_test)      # reuse those same numbers on TEST
```

Standardization re-centers and re-scales one column

Before: raw feature



After StandardScaler



Standardizing one feature: the shape of the distribution is kept, but it is re-centered on 0 and squeezed to unit variance.

NOTE – StandardScaler vs preprocessing.scale

scikit-learn also has a one-liner, `preprocessing.scale(X)`, that standardizes an array in place. It's fine for a quick look, but it can't be fit on the training set and reused on the test set, so it isn't leakage-safe. For a real workflow prefer `StandardScaler` (or a `Pipeline`).

Normalization (row-wise)

A **Normalizer** is different: it scales each **row** to unit length, working on a **cross-sectional** (row-by-row) basis rather than a column basis. With the default **L2 norm**, a row's value x becomes $x \div \sqrt{(x^2+y^2+z^2)}$; with the **L1 norm** the denominator is the sum of absolute values instead.

Why does normalization matter? Sometimes the signal lives in the *pattern across a row*, not in the raw magnitudes — the direction a vector points, not how long it is. Giving every row the same length lets the model compare like with like: two documents of very different length but the same word mix end up close together, and two customers with the same spending pattern but different totals look alike. It also keeps one huge-valued row from dominating distance calculations. That is why relative-proportion tasks lean on it.

```
import numpy as np
from sklearn.preprocessing import Normalizer

scaler = Normalizer().fit(X_train)          # default norm = 'l2'
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

As a **rule of thumb** the Normalizer often works well for **clustering** and **text classification**, where proportions matter more than absolute size, and less well for plain tabular classification like this one — here it scored about 52% versus about 64% for the StandardScaler. Treat it as a tool to try, not a guarantee.

TIP — Wrap the steps in a Pipeline

A Pipeline chains split-safe steps: Pipeline([('scale', StandardScaler()), ('pca', PCA(8)), ('clf', LinearSVC())]). Call fit() on the training data and every step is fit on train only — leakage handled for you.

6

Train and score a classifier

Now the model. A **Linear Support Vector Classifier (LinearSVC)** draws straight-line boundaries between the classes. We create it with default settings and **fit** it to the training data.

```
from sklearn.svm import LinearSVC
svc_clf = LinearSVC()
svc_clf.fit(X_train, y_train)

accuracy = svc_clf.score(X_test, y_test)
print(accuracy)          # 0.641... -> about 64%
```

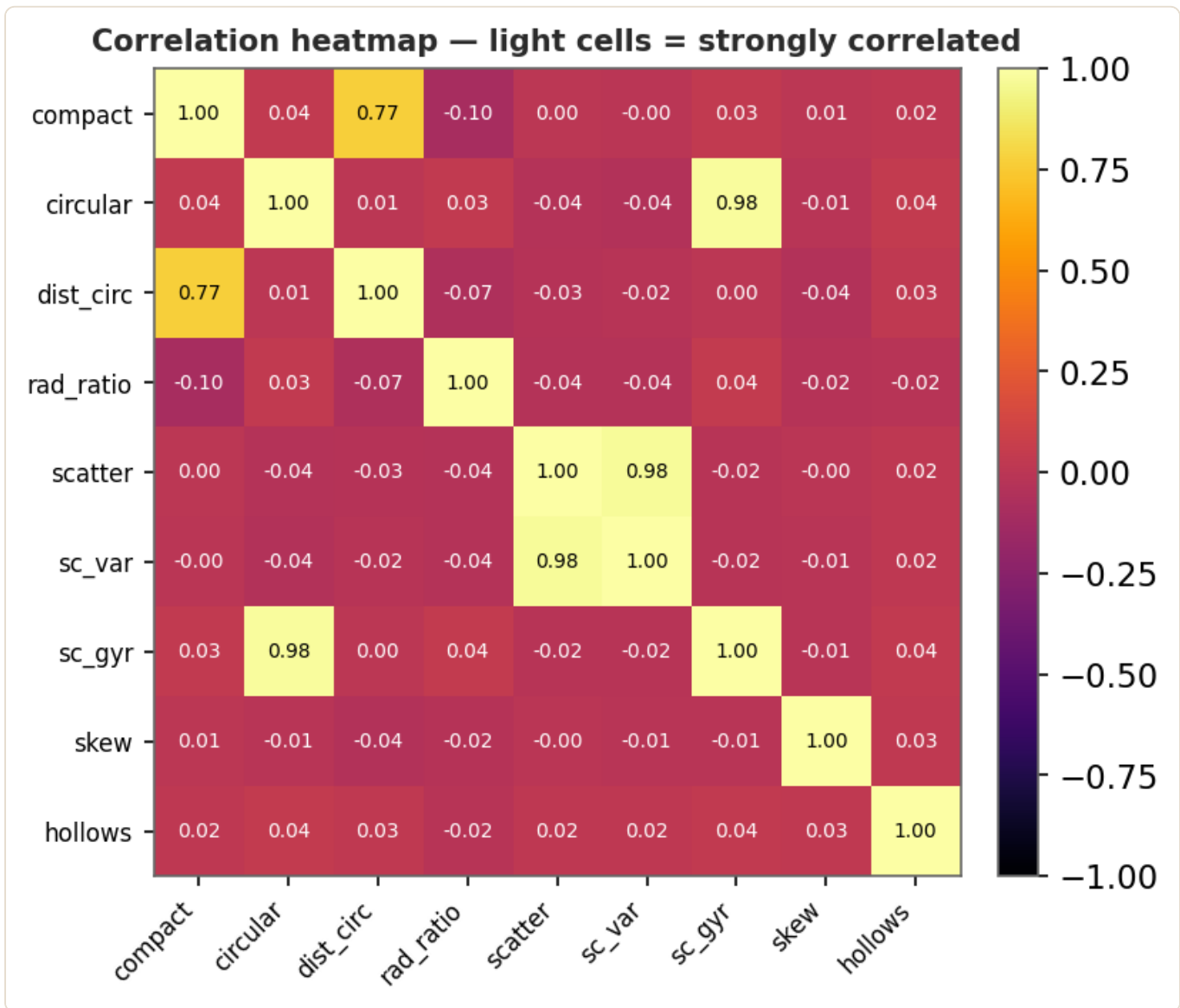
The **score** method makes predictions on `X_test`, compares them to the true `y_test` labels, and returns **accuracy** — the share it got right. At ~64% against a ~25% baseline, the model is doing real work. You can push further by **tuning parameters** such as the **tolerance** (`tol`) and the **penalty** in the **loss function**.

7 Spot correlated features

Many silhouette measurements say almost the same thing. Two features are **correlated** when they move together; the **correlation coefficient** runs from -1 (perfect **negative**) through 0 (none) to +1 (perfect **positive**). The DataFrame **corr** method builds a **correlation matrix** of every pair, and **Seaborn** turns it into a **heatmap** that's far easier to read.

```
import matplotlib.pyplot as plt
import seaborn as sns

corr = vehicle.corr()
plt.figure(figsize=(12, 7))
sns.heatmap(corr, vmax=.8, square=True, annot=True, fmt='.2f', cmap='inferno')
plt.show()
```



Bright cells flag strongly correlated pairs — e.g. scatter ratio and scaled variance, or circularity and radius of gyration.

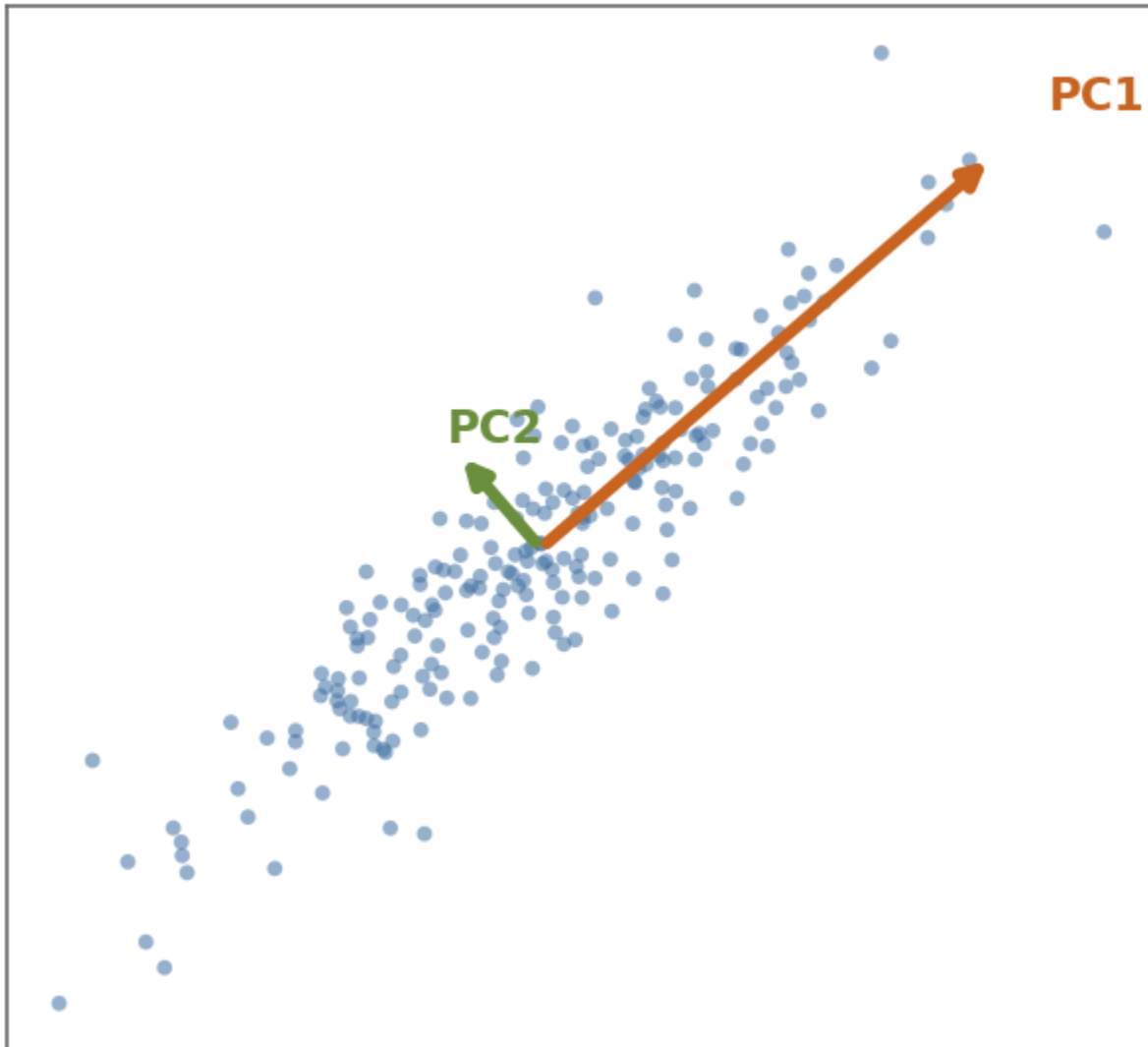
WATCH OUT — Don't just delete columns by hand

Dropping one of two correlated features is risky — you might throw away the column that best explains the outcome. PCA (next) is the safer fix.

Reduce dimensions with PCA

Principal Component Analysis is a **mathematical transformation** that turns many **correlated** features into a smaller set of new, **uncorrelated** features called **principal components**. Each component points along a direction of maximum spread in the data. Going from nine features to a handful of components is **dimensionality reduction**, and PCA lives in scikit-learn's **decomposition** module. As with scaling, fit it on the training data only.

PCA finds new axes along the data's variance



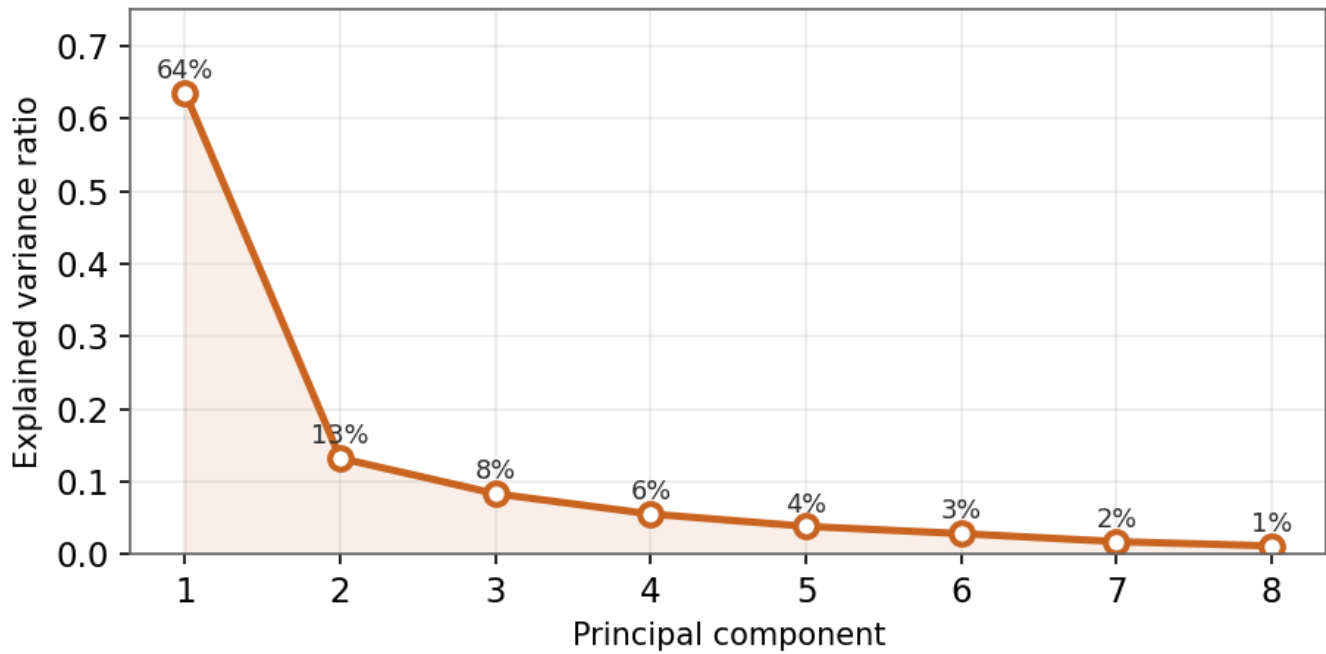
PCA finds new axes aligned with the data's variance. PC1 captures the most spread, PC2 the next, and so on.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=8, whiten=True)          # keep 8 components (an example choice)
X_train = pca.fit_transform(X_train)           # fit on train...
X_test = pca.transform(X_test)                 # ...then transform test

pca.explained_variance_                        # raw variance captured by each component
pca.explained_variance_ratio_                 # same, as a proportion (first ~0.64)
```

The number of components is a **choice, not a rule**. Here we drop from nine features to eight to shed the least-informative direction, but you could keep more or fewer. A common recipe is to keep enough components to reach a target cumulative **explained variance** — say 95%. Use `explained_variance_ratio_` to see what fraction of the total spread each component captures.

Scree plot — variance captured by each component



A scree plot of explained-variance ratio. The first component holds most of the information; later ones add little.

```
plt.plot(pca.explained_variance_ratio_)
plt.xlabel('Dimension'); plt.ylabel('Explained variance ratio')
plt.show()
```

Rebuilding the LinearSVC on 8 components gave the *same* ~64% accuracy as all 9 features — fewer inputs, equal performance, and a more **robust** model because the correlated noise is gone. Dropping to 6 components still held ~55%.

TIP — Whitening

`whiten=True` rescales the components to unit variance so they're fully uncorrelated — handy before feeding them to some models.

Two practice exercises

1 • Encode a wine target. A target column contains *red*, *white*, *rose*, *sparkling*. Because it's the **target** (not a feature), **label encoding** is fine and intuitive. `LabelEncoder` maps them alphabetically to red=0, rose=1, sparkling=2, white=3.

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y = le.fit_transform(['red', 'white', 'rose', 'sparkling'])
# alphabetical -> red=0, rose=1, sparkling=2, white=3
```

2 • Reason about the tools. Remember the four takeaways the exercise checks: the **Normalizer** works per row (unlike column scalers); **PCA** is worth it when you have dozens or hundreds of correlated features and want speed plus robustness; **train_test_split** beats hand-slicing arrays; and a Seaborn **heatmap** of `df.corr()` is the quick way to see which columns to reconsider.

WATCH OUT – Avoid data leakage (put into practice above)

This guide already splits before scaling and PCA, and fits both on the training set only. The safest way to guarantee that is a Pipeline, which fits every step on train and applies it to test automatically — no test information leaks into training.

QR Quick reference

The whole workflow at a glance.

Step / task	Tool or method	Library
Load a CSV into a table	<code>read_csv('vehicle.csv')</code>	pandas
Peek at first rows	<code>df.head()</code>	pandas
Drop unwanted columns	<code>df.drop([...], axis=1)</code>	pandas
Encode the text target	<code>LabelEncoder().fit_transform(y)</code>	scikit-learn
Split train / test (do this early)	<code>train_test_split(X, y, test_size=0.2)</code>	scikit-learn
Standardize columns (fit on train)	<code>StandardScaler().fit(X_train)</code>	scikit-learn
Normalize rows	<code>Normalizer().fit_transform(X)</code>	scikit-learn
Chain steps leak-free	<code>Pipeline([...])</code>	scikit-learn
Train a classifier	<code>LinearSVC().fit(X_train, y_train)</code>	scikit-learn
Score the model	<code>clf.score(X_test, y_test)</code>	scikit-learn
Correlation numbers	<code>df.corr()</code>	pandas
Correlation heatmap	<code>sns.heatmap(corr, annot=True)</code>	seaborn
Reduce dimensions	<code>PCA(n_components=8).fit(X_train)</code>	scikit-learn
Variance per component	<code>pca.explained_variance_ratio_</code>	scikit-learn

Every term from the study list, in plain language. Items marked (*dataset-specific*) are features or classes of the *Vehicle Silhouettes* dataset, not standard ML vocabulary.

Accuracy — Fraction of predictions the model got right; the headline score used in this guide.

Algorithm — A step-by-step recipe a computer follows to learn from or transform data.

Array — An ordered grid of numbers; NumPy's core data structure.

Array slicing — Selecting parts of an array with index notation, e.g. rows 0–5.

astype function — Pandas method that converts a column to another type, e.g. `astype(str)`.

Benchmark accuracy — A baseline score to beat; here 25% from guessing among four classes.

Blind guessing — Guessing the class with no information; sets the accuracy floor.

Bus class — (*dataset-specific*) One of the four vehicle types in the target column.

Categorical target — A label column whose values are named categories (bus, van...), not numbers.

Circularity — (*dataset-specific*) A silhouette shape feature describing how round the outline is.

Class column — The column holding the category each record belongs to (the target).

Classification — Predicting which category an example belongs to.

Classification model — A model trained to assign inputs to categories.

Clustering — Grouping unlabeled data by similarity; a task where the Normalizer suits well.

Column headers — The names given to each column of a DataFrame.

Column-wise scaling — Scaling each feature (column) on its own, as StandardScaler does.

Compactness — (*dataset-specific*) A silhouette feature measuring how tightly the shape is packed.

Continuous data — Numeric values that can take any value in a range, e.g. 3.7, 105.2.

Continuous features — Input columns holding continuous numeric values.

Correlated features — Features that move together, carrying overlapping information.

Correlation — How strongly two features move together.

Correlation coefficient — A number from -1 to +1 measuring that relationship.

Correlation matrix — A table of correlation coefficients for every pair of features.

Cross-sectional normalization — Same as **Row-wise normalization** — scaling each row on its own; what the Normalizer does.

CSV dataset — Data stored as comma-separated values in a text file.

Data preprocessing — Preparing raw data (scaling, encoding, cleaning) before modeling.

Data transformation — Changing data's form or scale to help the model.

Data visualization — Showing data graphically (e.g. heatmaps) to spot patterns.

DataFrame — Pandas' table of rows and columns.

Dataset — The full collection of records used to train and test.

Dataset trimming — Removing unneeded columns to simplify the data.

Decomposition — Breaking data into simpler parts; the scikit-learn module that holds PCA.

Dependent variable — The value being predicted (the target, y).

Dimensionality reduction — Cutting the number of features while keeping most of the information.

Distance circularity — (*dataset-specific*) A silhouette feature related to circularity measured via distances.

Distribution — The shape of how a feature's values are spread out.

drop function — Pandas method that removes rows or columns.

E-commerce recommendation — An ML use case: suggesting products to online shoppers.

Elongatedness — (*dataset-specific*) A silhouette feature for how stretched-out a shape is.

Encoding — Converting text categories into numbers a model can use.

Explained variance — How much of the data's spread each principal component captures (raw amount).

Explained variance ratio — The same, expressed as a proportion of the total variance.

Feature — One input column used to make predictions (an X column).

Feature correlation — Correlation measured between input features.

Feature extraction — Building new features (like principal components) from existing ones.

Feature scaling — Adjusting feature ranges so no single feature dominates by size.

Feature selection — Choosing a subset of existing features to keep.

fit method — Learns parameters from data — a scaler's mean, or a model's weights.

fit_transform method — Fits, then immediately applies the transformation, in one step.

Four-class classification — A problem with four possible target categories (as here).

get_params method — Returns the settings an estimator was configured with.

head function — Shows the first few rows of a DataFrame.

Heatmap — A colored grid; used here to visualize the correlation matrix.

Highly correlated variables — Features whose correlation is close to ± 1 .

Hollows ratio — (*dataset-specific*) A silhouette shape feature.

Independent variable — An input feature used to predict the target (an X column).

Input data — The features fed into the model (X).

Input feature — A single input column.

L1 norm — Row length as the sum of absolute values; a Normalizer option.

L2 norm — Row length as the square root of the sum of squares (Euclidean); Normalizer's default.

Label — The known answer/category for a record.

Label encoding — Turning each distinct category into an integer.

LabelEncoder — The scikit-learn tool that does label encoding (meant for the target y).

Linear support vector classifier — A classifier that separates classes with straight-line boundaries.

LinearSVC — scikit-learn's implementation of that classifier.

Longitudinal scaling — Scaling a column across all rows (column-wise); contrast with row-wise.

Loss function — The quantity a model minimizes while training.

Machine learning — Teaching computers to find patterns in data instead of hard-coding rules.

Machine learning model — The trained object that makes predictions.

Mathematical transformation — A formula-based change applied to data; PCA is one.

Matplotlib — Python's core plotting library.

Maximum length aspect ratio — (*dataset-specific*) A silhouette feature (one of the trimmed columns).

Mean — The average value of a feature.

Model evaluation — Measuring how well a model performs, usually on test data.

Model fitting — Training a model on data (calling `fit`).

Model parameter tuning — Same as **Parameter tuning** — adjusting a model's settings (tolerance, penalty, ...) to improve results.

Model prediction — The output a trained model gives for new inputs.

Model robustness — How well a model holds up on new, real-world data.

Model score — A single number summarizing performance (accuracy here).

Model training — The process of learning from the training set.

Multi-class classification — Classification with more than two categories.

Negative correlation — One feature rises as the other falls (coefficient below 0).

Normalization — Scaling each row to unit length.

Normalized array — A NumPy array after normalization.

Normalized DataFrame — A DataFrame holding normalized values.

Normalizer — The scikit-learn tool that normalizes rows.

Numeric data — Values stored as numbers; contrast with **String data**.

Numeric label — A category converted to an integer.

NumPy — Python's core numerical / array library.

Object data type — Pandas' type for text or mixed columns (why we cast with `astype(str)`).

One-hot encoding — Turning a category into several 0/1 columns; better for input features than for the target.

Pandas — Python's library for tables (DataFrames).

Parameter tuning — Adjusting settings like tolerance or penalty for better performance.

Penalty — A term that discourages overly complex models (regularization).

Positive correlation — Two features rise together (coefficient above 0).

Prediction — The category a model assigns to an input.

Preprocessing — Umbrella term for **Data preprocessing** — the preparation steps (scaling, encoding, cleaning) done before modeling.

Principal component — A new combined feature that captures a direction of variance.

Principal Component Analysis — A method that turns correlated features into fewer uncorrelated components.

Principal-axis aspect ratio — (*dataset-specific*) A silhouette feature (a trimmed column).

Principal-axis rectangularity — (*dataset-specific*) A silhouette feature (a trimmed column).

Probability baseline — The chance-level accuracy to beat — 25% for four equal classes.

Python — The programming language used throughout.

Radius ratio — (*dataset-specific*) A silhouette shape feature.

Random guessing — Picking a class at random; defines the baseline accuracy.

random_state — A seed that makes random steps (like the split) repeatable.

read_csv function — Pandas function that loads a CSV file into a DataFrame.

Record — A single row of the dataset (one example).

Row-wise normalization — Scaling each row on its own; what the Normalizer does.

Saab class — (*dataset-specific*) One of the four vehicle types.

Scaled data — Data after a scaler has been applied.

Scaled radius of gyration — (*dataset-specific*) A silhouette shape feature.

Scaled variance — (*dataset-specific*) A silhouette shape feature.

Scaler — A tool that rescales features, e.g. StandardScaler or Normalizer.

Scaling — Adjusting the range of values so features are comparable.

Scatter ratio — (*dataset-specific*) A silhouette shape feature.

scikit-learn — The main Python machine-learning library used here.

score function — Returns a model's accuracy on given data and labels.

Scree plot — A line plot of explained variance per component; helps you pick how many to keep.

Seaborn — A plotting library built on Matplotlib; used for the heatmap.

Sedan class — (*dataset-specific*) A vehicle-type category in the target column.

Silhouette data — (*dataset-specific*) The vehicle-outline measurements that form the features.

Skewness — (*dataset-specific*) A silhouette feature measuring asymmetry of the shape.

Standard deviation — A measure of spread; equals 1 after standardization.

Standard scaler — StandardScaler: gives each feature zero mean and unit variance.

Standardization — Rescaling a feature to zero mean and unit variance.

Stock-price prediction — An ML use case mentioned as motivation.

String data — Text values (like class names) that must be encoded.

Support Vector Classifier — A classifier that separates classes by a maximum-margin boundary.

Target column — The column being predicted (the label / y).

Target variable — What the model predicts — the target.

Test data — Held-out examples used only to evaluate.

Test set — The portion of data reserved for testing (20% here).

test_size — The `train_test_split` argument setting the test fraction.

Text classification — Classifying documents or text; a case where the Normalizer shines.

Tolerance — A stopping-threshold setting for the optimizer (`tol`).

Train-test split — Dividing data into training and testing portions.

train_test_split — The scikit-learn function that performs that split.

Training data — Examples the model learns from.

Training set — The portion used for training (80% here).

transform method — Applies a fitted transformation to data.

Uncorrelated variables — Features with little linear relationship; PCA outputs these.

Unique values — The distinct entries in a column, e.g. the four vehicle classes.

Unit variance — Variance of 1 after standardization.

Van class — (*dataset-specific*) A vehicle-type category.

Variance — How spread out a feature's values are.

Vehicle classification — (*dataset-specific*) Predicting vehicle type from silhouette features.

Vehicle silhouette dataset — (*dataset-specific*) The Statlog dataset used throughout the course.

Warning suppression — Turning off distracting warnings, e.g.

```
warnings.filterwarnings('ignore').
```

Whitening — A PCA option that makes outputs uncorrelated with unit variance.

Wine-class encoding — The exercise: label-encoding wine types (red, rose, sparkling, white).

X variable — The features / input (capital X).

Y variable — The target / output (y).

Zero mean — Average of 0 after standardization.

Beginner-friendly references, ordered to follow the workflow above.

Getting started with scikit-learn

Features (X), targets (y), estimators, preprocessing, fitting and prediction.

https://scikit-learn.org/stable/getting_started.html

Loading CSV data with pandas

Importing a CSV file into a DataFrame with `read_csv`.

https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html

Encoding target labels with LabelEncoder

Turn class names such as bus or van into numeric labels. Meant for y, not features.

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html>

Splitting data with train_test_split

Create training and test datasets.

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

Introduction to preprocessing

Overview of scaling, normalization and encoding techniques.

<https://scikit-learn.org/stable/modules/preprocessing.html>

StandardScaler

Standardize each feature to roughly zero mean and unit variance.

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>

Normalizer

Normalize each individual row using an L1, L2 or max norm.

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.Normalizer.html>

Computing correlations with DataFrame.corr

Produce a correlation matrix for numerical features.

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.corr.html>

Correlation heatmaps with Seaborn

Visually identify strongly correlated features.

<https://seaborn.pydata.org/generated/seaborn.heatmap.html>

Linear SVC (LinearSVC)

The linear support-vector classifier fitted in the notes.

<https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html>

Classification metrics & model evaluation

Accuracy, balanced accuracy, precision, recall, F1 and confusion matrices.

https://scikit-learn.org/stable/modules/model_evaluation.html

PCA and dimensionality reduction

Conceptual explanation of principal component analysis.

<https://scikit-learn.org/stable/modules/decomposition.html>

PCA API & explained variance

`n_components`, `explained_variance_`, `explained_variance_ratio_` and `whitening`.

<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>

Beginner PCA example (Iris dataset)

A complete, visual PCA walk-through.

https://scikit-learn.org/stable/auto_examples/decomposition/plot_pca_iris.html

Avoiding data leakage

Modern guidance: split first, then fit scaling and PCA on the training data only.

https://scikit-learn.org/stable/common_pitfalls.html

UCI Vehicle Silhouettes dataset

The Statlog classification dataset referenced in the notes.

<https://archive.ics.uci.edu/dataset/149/statlog%2Bvehicle%2Bsilhouettes>